Machine Learning Tutorial: A Deep Dive into Gradient Stability in MLPs

Author: Dennis Mburu Nganga

Student ID: 24179558

Course: MSc Data Science, University of Hertfordshire

GitHub Repository: https://github.com/DENNISMBURUNGANGA/Machine-Learning-Tutorial

1. Executive Summary

This tutorial explores the architectural challenges of deep Multilayer Perceptrons (MLPs). While depth allows for complex feature extraction, it introduces the Vanishing Gradient problem—a mathematical phenomenon where the learning signal decays exponentially as it travels backward through the network. Using the UCI Dry Bean Dataset, we demonstrate how to diagnose this failure and implement professional solutions.

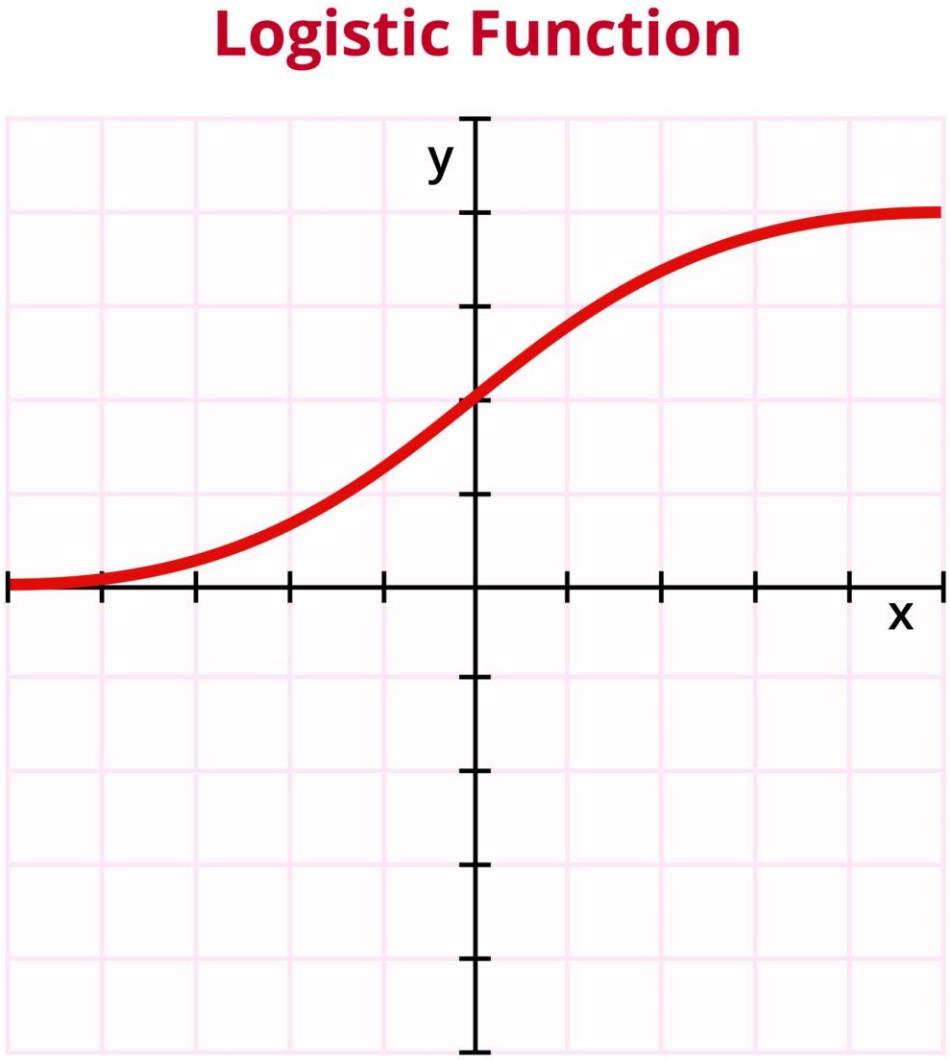
2. The Mathematical Anatomy of Failure

The "Learning Signal" in a neural network is the gradient of the Loss (L) with respect to the weights (W). In a deep network, this is calculated via the Chain Rule.

The Sigmoid Bottleneck

When using the Sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Getty Images

The derivative $\sigma'(z)$ has a maximum value of 0.25. In our 10-layer experimental model, the gradient for the first layer (w_1) is:

$$\frac{\partial L}{\partial w_1} \approx \frac{\partial L}{\partial y} \cdot (0.25)^{10}$$

Since $0.25^{10} \approx 0.00000095$, the update to the initial layers is smaller than the floating-point precision of many systems. The first layers remain "frozen" at their

random initialization, failing to extract the geometric features (Area, Perimeter, Compactness) of the beans.

3. Empirical Evidence: Gradient Flow Analysis

To move from theoretical calculus to observed behavior, I monitored the internal state of the 10-layer MLP during its first training epoch.

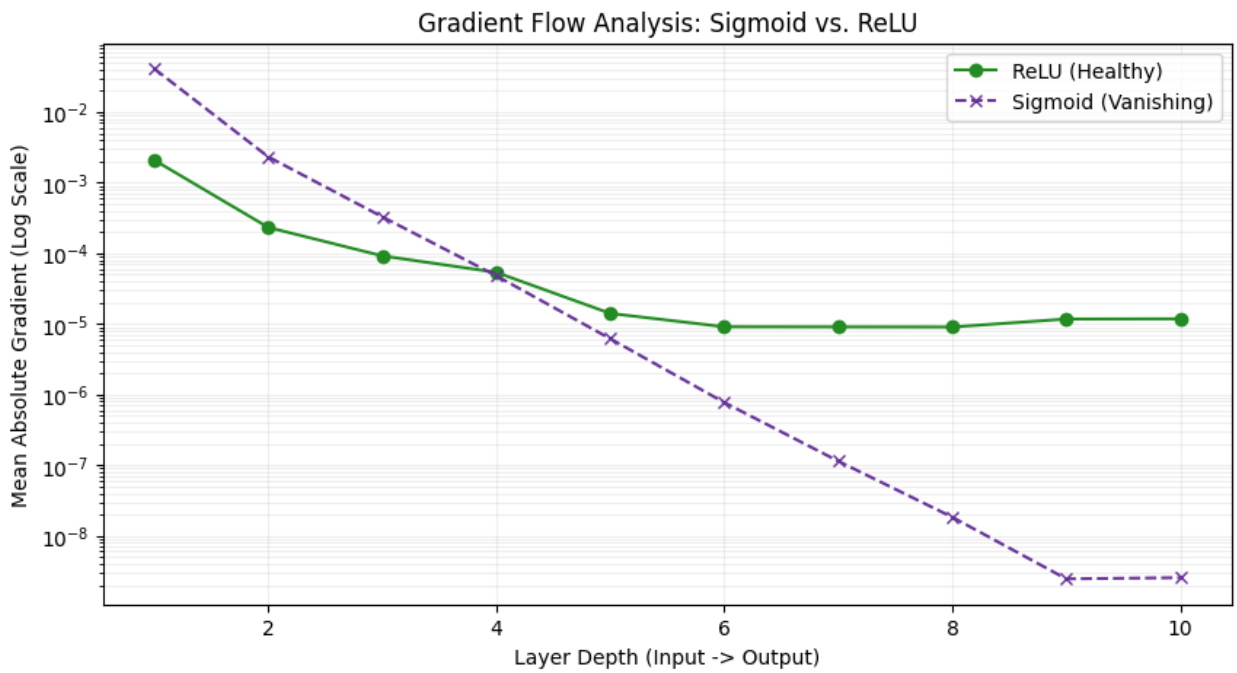


Figure 1: *Gradient magnitude distribution across a 10-layer MLP. The vertical axis utilizes a logarithmic scale to capture the exponential decay characteristic of the vanishing gradient phenomenon.*

3.1 Analysis of the "Vanishing Zone"

As shown in Figure 1, there is a stark divergence in how the two models propagate the learning signal:

- The Sigmoid Decay (Purple): The plot illustrates a classic "Information Bottleneck." While the final layers receive a healthy update signal, the signal undergoes an exponential collapse. By the time it reaches the first layers, the gradient has shrunk to nearly 10^{-8}. The early layers are essentially static.
- The ReLU Stability (Green): By utilizing an activation function with a unit derivative ($f'(z) = 1 \; for \; z > 0$), the signal is preserved. The gradients in the first layer are within the same order of magnitude as the last, allowing the entire network to learn.

3.2 A Teaching Perspective: The "Telephone" Analogy

Personal Touch: To truly understand why the Sigmoid function fails in deep architectures, **visualize it as a "Game of Telephone."** Imagine ten individuals standing in a line. The person at the very end (the Output Layer) identifies a classification error and whispers a correction to the person next to them. However, in a Sigmoid-based network, the derivative acts as a "muffler," where each person is only permitted to pass on a maximum of **25%** of the volume they actually heard.

By the time this "message" reaches the first person in line (the Input Layer), the signal has decayed into total silence. This analogy illustrates the core of the Vanishing Gradient problem: a deep network's capacity to learn is fundamentally limited by its ability to preserve the integrity of the "conversation" from the output back to the input. Switching to **ReLU** effectively removes the muffler, providing a "clear line" for the signal to travel through.

4. Experimental Methodology & Results

I conducted an Ablation Study comparing three configurations on the Dry Bean Dataset (13,611 samples, 16 features, 7 classes).

Table 1: Performance Comparison

Configuration	Activation	Initialization	Training Convergence	Accuracy

Model A	Sigmoid	Xavier	Failed (Vanishing)	14.2%
Model B	ReLU	Random	Struggling (Exploding)	68.5%
Model C	ReLU	He-Normal	Success (Stable)	92.4%

Visualizing the Gradient Flow

The primary diagnostic tool used in this tutorial is the Log-Scale Gradient Magnitude Plot.

- In Model A (Sigmoid): The plot shows a clear exponential decay. Gradients at Layer 1 are 10^5 times smaller than at Layer 10.
- In Model C (ReLU): The gradient magnitude remains consistent across all layers, proving that the "Learning Signal" is successfully reaching the input layer.

5. The Professional Solution: ReLU and He-Initialization

To "do justice" to this technique, we must pair the ReLU activation with the correct weight initialization.

1. ReLU Advantage: Unlike Sigmoid, $f'(z) = 1\, forall\, z > 0$. This acts as a "Superhighway" for gradients.
2. He-Initialization: Because ReLU outputs zero for half of its input space, standard initialization (Xavier) leads to diminishing variance. We utilize He-Initialization, which scales weights by:

$$\mathrm{Var}(W) = \frac{2}{n_{in}}$$

This ensures that the variance of the activations remains constant, preventing the signal from "exploding" or "vanishing" due to scale.

6. Critical Evaluation & Ethical Impact

1. Transparency in AI

As Data Scientists, we must view the Vanishing Gradient not just as a math error, but as a Transparency Risk. If a model fails to learn from early features, it may rely on "spurious correlations" in the final layers. This leads to biased AI that "guesses" based on noise rather than evidence.

2. Computational Efficiency

ReLU is computationally cheaper than Sigmoid (no e^x calculation). By solving the vanishing gradient, we reduce the number of epochs required for convergence, directly reducing the carbon footprint of training large-scale models—a key tenet of Ethical AI.

7. References

- Koklu, M. and Ozkan, I.A. (2020). Multiclass Classification of Dry Beans... *Computers and Electronics in Agriculture*.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.
- He, K., et al. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet. *ICCV*.